PS13 — Air-Gapped AI Predictive Co-Pilot

Project Summary and Team Briefing Document — Smart India Hackathon, Institute Level Round

Problem Statement: PS13 | Organisation: ISRO | Category: Software

This document is a working reference for the team ahead of the institute-level round. It lays out the problem in plain terms, why it is a genuine and unsolved problem rather than a routine engineering task, what we are proposing to build, the technology choices behind it and why each was made, what makes our approach different from existing tools, and why the approach holds up beyond the hackathon demo. Read this before the pitch so that every team member can answer follow-up questions consistently.

1. What Is the Problem

ISRO operates mission-critical MPLS networks that connect ground stations, control centres and data centres. For national security reasons, these networks are air-gapped — they cannot touch the public internet or any cloud service, not even for monitoring or analytics. A single external connection is treated as an attack surface that a space agency cannot afford.

Today, when something goes wrong on this network, a human engineer has to manually dig through SNMP and syslog data, cross-reference paper or PDF runbooks, and diagnose the issue under time pressure. There is no predictive layer and no natural-language assistant — everything is reactive, manual and dependent on individual experience.

Who feels this problem

- Network Operations Centre (NOC) engineers and on-call administrators inside air-gapped environments — ISRO ground segment teams, defence network operators, and banking infrastructure teams under data-localization rules.
- Alert fatigue from raw SNMP/syslog floods with no prioritization.
- Tribal knowledge — only senior engineers instinctively know what a given error pattern means.
- Runbooks exist only as static PDFs or Word files that are not searchable in the moment of an incident.
- Failures are diagnosed after they cause downtime, never predicted ahead of time.

Why the problem exists

There is a direct tension between two facts. Modern AI capability — large language model reasoning, retrieval-augmented generation, predictive analytics — is almost always delivered as a cloud API. High-security environments such as ISRO are contractually and legally barred from sending any data outside their perimeter. Most "AI-powered NOC tools" on the market (ServiceNow AIOps, Cisco AIOps, Splunk ITSI) assume cloud connectivity for their AI backend. There is no mature, widely-adopted, fully local AIOps copilot product today — this is a real, unsolved gap and not just an ISRO-specific inconvenience.

> *Core framing to remember: the challenge is not "build an AI for logs." It is "build a trustworthy, explainable reasoning system that gets zero help from the cloud and must prove every claim it makes against real evidence" — because in a security-critical network, an AI that is occasionally brilliant but unverifiable is more dangerous than no AI at all.*

Why existing tools do not solve it

Category of tool	Examples	Why it falls short for PS13
Cloud AIOps platforms	Splunk ITSI, Moogsoft, Datadog AIOps	Strong capability, but the AI/ML inference layer runs on vendor cloud servers — a hard no in an air-gapped network.
Traditional rule-based NMS	Nagios, Zabbix, SolarWinds	Works fully offline but is purely reactive and threshold-based. No prediction, no natural-language explanation, and rules need constant manual tuning.
On-prem AIOps suites	Various enterprise vendors	Exist, but are expensive, closed-source, not customizable to ISRO's own runbooks, and behave as an unexplainable black box — hard to trust in a security context.

2. What We Are Building

Our solution is an **air-gapped AI predictive co-pilot**: a system that watches network logs continuously, predicts failures before they happen, and explains every recommendation in plain English — grounded in the organisation's own runbooks — while running entirely on local hardware with zero outbound network calls.

End-to-end workflow

- **Ingest** — SNMP traps and syslog streams are collected from the network and normalised.
- **Detect** — an anomaly detection model flags unusual patterns in the incoming data.
- **Predict** — a failure-prediction step estimates the probability and urgency of an impending fault.
- **Retrieve** — a local retrieval-augmented generation (RAG) layer pulls the most relevant sections from the organisation's own runbooks and past incidents.
- **Explain** — a local language model turns the anomaly, prediction and retrieved evidence into a plain-English explanation.
- **Recommend** — the system proposes a concrete next action, with a confidence score and a citation to the exact source it used — never an unexplained black-box answer.

This maps directly onto the workflow the problem statement itself describes: logs → anomaly detection → failure prediction → local RAG → local LLM reasoning → explanation → recommendation. That alignment matters — it lets evaluators map our demo directly onto the brief instead of us re-interpreting the problem.

Primary users

Role	What they do with the system
NOC Engineer (primary user)	Views the live dashboard, asks the copilot questions in natural language, and acts on its recommendations.
Security / Audit Officer	Reviews the full audit trail of every AI decision for compliance.
Admin	Uploads and manages runbooks, configures alert thresholds.

3. What We Are Using to Solve It, and Why

Every component below was chosen specifically because it can run fully offline on modest, on-prem hardware — not a GPU cluster — while still being mature and well documented enough to build reliably in a short timeframe.

Layer	Technology	Why this choice
Log parsing	Drain3	A lightweight, proven online log-template mining algorithm, purpose-built for exactly this kind of noisy, multi-format SNMP/syslog data.
Anomaly detection	Isolation Forest / scikit-learn (PyOD, Prophet as extensions)	Mature, well-documented anomaly detection that does not require a custom deep-learning pipeline or labelled failure data to get started.
Vector store (RAG)	ChromaDB (FAISS as fallback)	A Python-native vector database that runs entirely locally and gives high-accuracy similarity search over runbooks and past incidents.
RAG orchestration	LangChain	Saves significant development time versus hand-rolling retrieval logic, and keeps the retrieval step auditable.
Local language model	Ollama serving Llama 3 / Mistral (7B–8B class)	Ollama abstracts local model serving behind a simple local API, which reduces setup risk. Model sizes were deliberately chosen to fit modest on-prem hardware rather than assume a GPU farm.
Backend	FastAPI (Python)	Async support matters for LLM inference latency, and it integrates the ML/RAG pipeline natively without a language bridge.
Structured data store	PostgreSQL	ACID-compliant relational storage for incidents, logs and the audit trail — a good fit for the relationships between incidents, log events and recommendations.
Frontend	React	A component-based dashboard and chat interface; no server-side rendering or SEO need exists for an internal NOC tool, so a lightweight setup is enough.
Deployment	Docker	Containerising the whole stack is the clearest way to prove and demonstrate offline deployability on a single on-prem server.

Deliberately **not** used: any cloud provider (AWS, Azure, GCP) — their absence is the entire point of the problem statement, not a gap in our stack. Kubernetes was also left out: it is overkill for a single-node, air-gapped deployment and would only hurt credibility with evaluators who understand the constraint.

Why explainability was prioritised over raw model size

Small local language models (7B–8B parameters, the class that can realistically run on modest on-prem hardware) are noticeably weaker at structured reasoning than large cloud models. Naive prompting on a small model produces confident-sounding but wrong explanations — this is the single biggest technical risk in the entire problem space. Our answer is to lean on RAG grounding rather than raw model

intelligence: every explanation must cite a specific runbook or log excerpt, and if no relevant evidence is found, the system says so explicitly instead of guessing. A bigger model does not solve this on its own — grounding does.

4. Why This Is Necessary

- **The constraint is non-negotiable.** ISRO's networks cannot connect to any cloud service, by policy and by security requirement — this rules out essentially every commercial AIOps product on the market today.
- **The manual process does not scale.** Alert fatigue, static PDF runbooks and tribal knowledge mean diagnosis happens after downtime, not before it, and depends heavily on which engineer is on shift.
- **The market gap is real, not invented.** There is no dominant, well-known open-source or commercial product specifically built as an air-gapped AIOps copilot — most AIOps innovation today is cloud-first by default.
- **The same constraint applies well beyond ISRO.** Defence networks, RBI-regulated banking infrastructure under data-localization mandates, and power-grid SCADA systems all face an almost identical "no cloud AI" restriction.

5. What Makes Our Solution Unique

Our differentiation is not "an AI chatbot for logs." It is a specific, defensible design philosophy: **every claim the system makes must be traceable to real evidence**, because in a security-critical network an unverifiable AI answer is worse than no AI at all.

USP	What it means in practice
Zero cloud dependency	The entire pipeline — ingestion, detection, prediction, retrieval, reasoning and the dashboard — runs inside the secure perimeter with no outbound network call at any point.
RAG-grounded, verifiable answers	Every recommendation cites the exact runbook section or past incident it came from, with a confidence score, instead of an unexplained model output.
Genuinely predictive, not reactive	Failures are flagged before they cause downtime, unlike threshold-based tools like Nagios or Zabbix which only react after a limit is crossed.
Full audit trail	Every prediction, retrieval and recommendation is logged and timestamped, reviewable by a dedicated auditor role — built for a compliance-heavy environment from day one.
Vendor-neutral, open-component stack	Built entirely from open, auditable components (Ollama, ChromaDB, FastAPI) rather than a closed commercial black box, so ISRO could inspect or modify any layer.
Graceful degradation	If the language model or vector database becomes unavailable, the system falls back to rule-based alerting rather than failing silently — important for a tool that cannot afford to just stop working.

Against named alternatives specifically: cloud AIOps tools (Splunk ITSI, Moogsoft, Datadog) are architecturally excluded by the air-gap requirement; classic NMS tools (Nagios, Zabbix, SolarWinds) have no prediction and no natural-language reasoning; enterprise on-prem AIOps suites exist but are closed,

expensive and produce recommendations that cannot be audited or trusted in a security context. Our approach is the only one that is simultaneously offline, predictive, explainable and auditable.

6. How It Is Sustainable

Operationally sustainable

- No internet is required at runtime, so there is no live threat-intelligence feed or cloud API to depend on. The system is self-contained by design.
- Model and knowledge-base updates are delivered through signed offline update packages over approved removable media — the same pattern already used for patching in other air-gapped environments, so it fits existing security procedures rather than inventing a new risk.
- If a runbook becomes outdated, an admin simply re-uploads it and the vector store re-indexes immediately; the language model itself does not need retraining, which keeps maintenance lightweight.

Financially sustainable

The cost structure is dominated by a one-time on-prem server purchase and open-source software with no license fees. There is no per-API-call cloud billing, unlike commercial AIOps platforms, which makes ongoing operating cost predictable and low.

Category	Item	Estimated cost
Compute hardware	On-prem server, 8–16GB RAM, optional GPU (one-time)	Rs 80,000 – 2,00,000
AI models	Local LLM (Llama 3 / Mistral) and embeddings	Free, open-source license
Software stack	FastAPI, React, ChromaDB, PostgreSQL, Docker	Free, open-source
Maintenance	Offline update cycles via signed packages	Low — no per-API cloud billing

Scalable and extensible

- The backend is stateless and can be scaled horizontally within the secure network; the vector database and LLM inference are the main bottlenecks, and both can be scaled with additional on-prem compute or model quantization.
- The same architecture transfers directly to other air-gapped or data-localized sectors — defence networks, banking infrastructure under RBI mandates, and power-grid SCADA systems — without redesign.
- A future self-learning loop is possible without ever leaving the air-gap: engineers can confirm or correct a diagnosis, and that feedback is written back into the local knowledge base, letting the system improve over time on nothing but its own deployment's data.

7. Anticipated Evaluator Questions

A few questions come up in almost every round for a problem statement like this. Everyone on the team should be able to answer these consistently.

Likely question	Our answer
How do you run an LLM with zero internet?	Ollama serves the model locally through a local API. Model weights are installed once, offline; after that, zero network calls happen at runtime. We can demonstrate this by disconnecting the network live.
How do you update the model without internet access?	Through signed offline update packages delivered via approved removable media, following the same pattern used for patching in other secure environments.
What if the model hallucinates a wrong recommendation?	Every claim must cite a specific runbook excerpt. If no relevant evidence is found, the system says so explicitly instead of guessing, and every recommendation still requires human confirmation before any action is taken.
Where did you get real ISRO network data?	We did not, and should not, for security reasons. We used synthetic SNMP/syslog data plus the LogHub academic dataset to validate the anomaly-detection approach.
Why a local model instead of a bigger cloud one?	That is the entire premise of the problem statement — cloud AI is categorically disallowed in this environment. We chose the largest local model that is still fast enough for real-time NOC use.
How is this different from Splunk or Nagios?	Splunk's AI features depend on the cloud; Nagios is purely rule-based with no natural-language reasoning. We combine predictive machine learning with grounded, explainable natural language, fully offline.

8. One-Line Summary to Keep in Mind

> *An offline AI co-pilot that predicts MPLS network failures before they happen and explains every recommendation in plain English — grounded in the organisation's own runbooks, with zero internet access, ever.*

Prepared as a team reference for the Smart India Hackathon institute-level round — PS13, Air-Gapped AI Predictive Co-Pilot.